Exact Synthesis of Two-Qubit Clifford+CS Circuits with a Near-Minimal Number of K and CS Gates

Xiaoning Bian* and Yuan Feng†
Department of Computer Science and Technology, Tsinghua University
(Dated: May 13, 2026)

We give a method for the decomposition of an arbitrary two-qubit Clifford+CS operator (equivalently a 4×4 unitary with entries in $\mathbb{D}[i] = \mathbb{Z}[\frac{1}{2}, i]$, the Gaussian integers with dyadic denominators), into a Clifford+CS circuit. Given unitary U, let k be the least natural number such that entries of $(1+i)^k U$ are all Gaussian integers, then this method decomposes U, upto a global phase, into a circuit of length at most $20k+9$, using at most $2k$ K (i.e. a scaled H) gates and at most $2k+1$ Controlled-S gates, without using any ancilla. Moreover, the number of K gate is at most 2 times worse than optimal, the number of *CS* gate is at most 4 times worse than optimal. In practice, this method always acheives the optimal CS-count, and roughly the K-count is $k/2$ smaller than the best known, acheiving roughly a 25% K-count reduction. This method is similar to the work of Russell [38] for the Clifford+T gate set but with a more concrete bound analysis (e.g. bound 20k+9 v.s. $O(k)$), where the ring in use is $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ which is more complicated than and contains the ring here $\mathbb{Z}[\frac{1}{2}, i]$.

I. INTRODUCTION

In quantum computation, the decomposition of unitary operators into gates from a fixed universal set is an important problem [36]. In this paper we focus on the exact synthesis 2-qubit Clifford+Controlled-S. Previous work on multi-qubit synthesis of Clifford+CS operators or Clifford+T operators mainly focus on minimizing the use of CS gate or T gate since they are more expensive than Clifford operators in certain scenerio. But works on T gate optimization [1, 5, 8, 9, 22, 26] suggested that the H gate is an "obstruction" to their T-optimization method. Quoting from [1] summarizes the reason: "It has been shown that T-depth optimizations can be implemented efficiently for circuits consisting only of T and CNOT gates and that H gates impede the optimization significantly".

We propose a method that produces circuits with a near-minimal number of the K (i.e., a scaled H) gates and CS gates, hence alleviate such "obstruction". Synthesis problem that aims to minimize the use of two different gates simultaneously is also of mathematical interest. One of the reason is that previous work on single-qubit synthesis of Clifford+T operators acheieved the minimum use of Hadamard and T gates [15, 28, 32], an extension of which to multi-qubit case has been an open problem.

A. Background

Consider the following unitary operators:

$$i, \quad K = e^{-i\pi/4}H = \frac{1}{1+i}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}, \quad S = \begin{bmatrix}1 & 0\\ 0 & i\end{bmatrix},$$

$$CZ = \begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & -1\end{bmatrix}.$$

Here, i is a scalar, namely the usual complex unit. H is the Hadamard gate, and K is a scaled version of the Hadamard gate. S is sometimes called the *phase gate*, and CZ is the controlled-Z gate. When closed under multiplication, identities, and tensor products, these operators generate the *Clifford group* (possibly up to scalars, depending which scalars are included in the Clifford group — for our purposes, the scalars ± 1 and $\pm i$ are sufficient).

Every operator U obtained in this way is of size $2^n \times 2^n$ for some natural number n, and as usual, we say that U is an operator on n qubits. We write $\mathcal{CLI}(n)$ for the group of n-qubit Clifford operators (just $\mathcal{CLI}$ if $n = 2$). It is well-known that this group is finite for any given n (see, e.g., [39]), and therefore not universal for quantum computing. We can obtain a universal gate set by adding the controlled phase gate

$$CS = \begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & i\end{bmatrix}.$$

The resulting operators are called the Clifford+CS operators. Note that $CZ = CS^2$. The set of 2-qubit Clifford+CS operators is denoted by $\mathcal{CCS}$. The set of 2-qubit Clifford+CS circuits (a circuit is a finite sequence of gates) is denoted by $\mathcal{CIR}$. Each circuit $D \in \mathcal{CIR}$ implements a unique operator $[D] \in \mathcal{CCS}$.

We use the following notations: we write $K_0 = K \otimes I, K_1 = I \otimes K$, and similarly for S_0 and S_1 where I is the 2×2 identity operator. We identify the scalar i with the 4×4-matrix $i\,(I \otimes I)$. In the notation for controlled gates, we use the convention that the target is the last index. For example, CX_{01} is a controlled X-gate

* bian@mail.tsinghua.edu.cn
† yuan_feng@tsinghua.edu.cn

with control qubit 0 and target qubit 1, also shorted as CX. CX_{10} is shorted as XC. The controlled S- and Z-gates are symmetric, in the sense that $CS_{jk} = CS_{kj}$ and $CZ_{jk} = CZ_{kj}$, and therefore the order of indices does not matter for them, so we just write CS and CZ. We also use the Z-gate, the X-gate, the controlled K-gate $CK = I \oplus K$, and the swap gate, denoted by Ex. They are related to the "primitive" gates introduced so far by $Z = S^2$, $X = KZKi$, $CX = K_1 CZ K_1 \cdot i$, $Ex = CX\, XC\, CX$, and

$$CK = CZ \cdot Z_0 S_0 Z_1 S_1 K_1 CS K_1 S_1 \cdot i. \tag{1}$$

Similar to CX gate, CK_{01} is shorted as CK and CK_{10} is shorted as KC. By Equation 1, $CK \in \mathcal{CCS}$.

B. Related work

Operator synthesis can be done exactly or approximately. Here we only list a few of the closely related ones on exact synthesis.

1. Single-qubit and single-qudit

For single-qubit synthesis of Clifford+T operators, works [15, 28, 32] achieved the minimum use of both Hadamard and T gate. The work [17, 37] achieved the minimum use of T gate for single-qutrit synthesis of Clifford+T operators. The work [24] considered the single-qudit synthesis of Clifford+T operators. Forest et al. [11] generalized exact synthesis from Clifford+T to single-qubit Clifford-cyclotomic gate sets. Kliuchnikov and Yard [30] developed a general algebraic framework for exact synthesis over number rings, unifying several gate sets under one theory.

2. Two-qubit

We list two works on two-qubit synthesis that guaranteed T-optimality or CS-optimality: 2-qubit Clifford+T synthesis [31] and the work on 2-qubit Clifford+CS synthesis [18] synthesis. The first gave an algorithm that is near T-optimal (the worst case uses at most 10 times the minimum possible T gates). The latter gave an algorithm that is CS-optimal; they didn't mention the H-count, but we can deduce from their work that their H-count enjoys the same property as our method: the K-count is at most 2 times worse then optimal. Glaudell et al. [16] later gave a high-performance exact synthesis framework for two-qubit Clifford+T circuits that optimizes T-count exactly, using meet-in-the-middle with algebraic canonicalization and lookup tables. The above 3 works depends on the exceptional isomorphism between two Lie groups $SU(4) \cong \mathrm{spin}(6)$. Our work is about the CS-optimality and H-optimality of 2-qubit Clifford+CS synthesis, but without using this exceptional isomorphism. Our work use the similar "residue" analysis method as in Russell [38], but over a simpler ring $\mathbb{Z}[\frac{1}{2}, i]$ contained in their ring $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$. Given a unitary U, Russell's method output circuit of gate count $O(k)$, where k is the least natural number such that entries of $(1+\omega)^k U$ are in the ring $\mathbb{Z}[\omega]$.

3. Multi-qubit and multi-qutrit

Multi-qubit synthesis of Clifford+T operators is solved by Giles and Selinger [14] using at most 1 ancilla. The algorithm's worst case gate count is $O\left(3^{2^n} nk\right)$. Later work [27] improved the gate count to $O\left(4^n nk\right)$. Glaudell et al. later solved the exact multi-qubit synthesis problem of several subclasses of Clifford+T operators [4] such as the class of operators generated by $\{X, CX, CCX, H\}$, known as the Toffoli+Hadamard operators. Amy et al. adapted the method used in [27] to the Toffoli+Hadamard operators achieving a similar improvement on gate count [3]. All the works on multi-qubit synthesis introduced so far said nothing about T-count, let alone T-optimality. Kliuchnikov and Schönnenbeck [29] recently developed new optimal and heuristic algorithms for multi-qubit exact synthesis using A^* search instantiated with Hermitian lattice theory over number fields, achieving runtime proportional to the T-count for two-qubit circuits. The work [2, 10] considered exact synthesis of multi-qubit Clifford+Cyclotomic operators. The work [20, 25] considered exact synthesis of multi-qutrit Clifford+Cyclotomic operators. The work [19] proved "qutrit metaplectic gates are a subset of Clifford+T". The works [12, 13, 33–35] proposed heuristic algorithms that acheive some kinds of non-Clifford gate optimality.

II. SOME ALGEBRA

A. Rings

As usual, we write $\mathbb{N} = \{0, 1, \ldots\}$ for the set of natural numbers. We also write $\mathbb{Z}$, $\mathbb{R}$, and $\mathbb{C}$ to denote the rings of integers, real numbers, and complex numbers, respectively. We write $z^\dagger$ for the complex conjugate of a complex number. Recall that in any ring, a *unit* is an invertible element.

Let $\mathbb{Z}[i] = \{a + bi \mid a, b \in \mathbb{Z}\}$ be the ring of Gaussian integers. It is a Euclidean domain[6], and therefore, division with remainder, greatest common divisors, divisibility, and the concept of a prime (i.e. a non-unit that cannot be written as the product of two non-units) all can be used in $\mathbb{Z}[i]$. Let $\gamma = 1 + i$. It is well-known that γ is a Gaussian prime [23]. Note that γ is a divisor of 2, and γ^2 and 2 divide each other. Using Euclidean division, it is further easy to check that

$$\mathbb{Z}[i]/(\gamma) = \{0, 1\}, \quad \mathbb{Z}[i]/(2) = \mathbb{Z}[i]/(\gamma^2) = \{0, 1, i, 1+i\},$$

$$\text{and} \quad \mathbb{Z}[i]/(\gamma^3) = \{\pm 1, \pm i, 0, 1+i, 1-i, 2\}.$$

Note that the above is a slight abuse of notation. Normally, $\mathbb{Z}[i]/(\gamma)$ denotes the quotient ring of $\mathbb{Z}[i]$ modulo the ideal (γ) whose elements are residues nomrally denoted by $x \bmod(\gamma)$ for any $x \in \mathbb{Z}[i]$. Here we omit the mod (γ) for simlicity. Note that $\{0,1\}$ etc. are complete sets of residue classes and also the reminders from the Euclidean division of x divided by γ.

Let $\mathbb{D} = \mathbb{Z}[\frac{1}{2}] = \left\{\frac{a}{2^k} \mid a \in \mathbb{Z}, k \in \mathbb{N}\right\}$ be the ring of *dyadic fractions*, i.e., fractions whose denominator is a power of 2. Consider $\mathbb{D}[i] = \{r + si \mid r, s \in \mathbb{D}\} = \mathbb{Z}[\frac{1}{2}, i]$. For every $t \in \mathbb{D}[i]$, there exists some natural number k such that $\gamma^k t \in \mathbb{Z}[i]$. This motivates the following definition.

Definition II.1. Let $t \in \mathbb{D}[i]$. A natural number $k \in \mathbb{N}$ is called a *denominator exponent* for t if $\gamma^k t \in \mathbb{Z}[i]$. The least such k is called the *least denominator exponent* of t, denoted by $\mathrm{lde}_\gamma(t)$. Equivalently, the least denominator exponent of t is the smallest $k \in \mathbb{N}$ such that t can be written in the form $\frac{s}{\gamma^k}$, where $s \in \mathbb{Z}[i]$.

More generally, we say that k is a denominator exponent for a vector or matrix if it is a denominator exponent for all of its entries. The least denominator exponent for a vector or matrix is therefore the least k that is a denominator exponent for all of its entries.

Remark II.2. By enumerating all 2-qubit Clifford operators, we have that the lde of a 2-qubit Clifford operator is at most 2.

B. Residue rings

It is well-known that

$$\mathbb{Z}[i]/(\gamma^n) = \left\{ \sum_{i=0}^{n-1} b_i \gamma^i \bmod (\gamma^n) \,|\, b_i \in \{0,1\} \right\}.$$

We give an elmentary proof by induction on n. Note that similar proof is used for proving the case for $\mathbb{Z}[\sqrt{2}]/(\sqrt{2}^n)$ by Clement[7]. When $n = 1$, the claim obviously holds. Suppose this is true for n, i.e., for any $x \in \mathbb{Z}[i]$, x is congruent to some $\sum_{i=0}^{n-1} b_i \gamma^i \bmod (\gamma^n)$, that is

$$x = \sum_{i=0}^{n-1} b_i \gamma^i + k\gamma^n \text{ for some } k.$$

Since $k = b_n + k'\gamma$ for some $b_n \in \{0,1\}$, substituting k in and some rewrite gives

$$x = \sum_{i=0}^{n} b_i \gamma^i + k'\gamma^{n+1}, \text{ i.e., } x \equiv \sum_{i=0}^{n} b_i \gamma^i \bmod (\gamma^{n+1}).$$

For simplicity, we omit the mod (γ^n) in $\sum_{i=0}^{n-1} b_i \gamma^i \bmod (\gamma^n)$. Since $b_i \in \{0,1\}$, we write binary string $(b_0 b_1 ...)_2$ for the summation, and the parenthesis and subscript $_2$ can be omitted if we can infer that $b_0 b_1 ...$ are residues in binary format from the context.

Moreover, residue classes are distinct, i.e. if $b_0 b_1 ... b_{n-1}$ and $b'_0 b'_1 ... b'_{n-1}$ are equivalent, i.e., $\sum_{i=1}^{n-1} b_i \gamma^i - \sum_{i=1}^{n-1} b'_i \gamma^i = k\gamma^n$ for some k, then $b_i = b'_i$. Modding (γ) on both side gives $b_0 = b'_0$. Then cancelling b_0 and b'_i, and dividing both side by γ give

$$\sum_{i=1}^{n-1} b_i \gamma^{i-1} - \sum_{i=1}^{n-1} b'_i \gamma^{i-1} = k\gamma^{n-1}$$

Then modding (γ) on both side agian gives $b_1 = b'_1$. Repeat this process proves $b_i = b'_i$.

We write $\rho_n(x)$ for the binary residue $x \,(\bmod\ \gamma^n)$. From the proof, we see $\rho_n(x)$ is "stable" w.r.t increase of n, i.e., the first n digits of $\rho_{n+1}(x)$ is $\rho_n(x)$. Also ρ_2 is "stable" under taking complex conjugation, i.e., $\rho_2(x^\dagger) = \rho_2(x)$.

We sometimes also write $x \equiv_{\gamma^n} y$ as an abbreviation for $x \equiv y \bmod \gamma^n$. Modulo γ^3, we have

x	$\rho_3(x)$	x	$\rho_3(x)$	x	$\rho_3(x)$	x	$\rho_3(x)$
1	100	-1	101	i	111	$-i$	110
0	000	$1+i$	010	$1-i$	011	2	001

The $(\bmod\ \gamma^n)$ residue classes form a commutative ring and there is a canonical ring homomorphism from $\mathbb{Z}[i]$. In particular, modulo γ^2, the addition of the ring coincides with the bitwise exclusive or (denoted by $\oplus$) operation; $(10)_2$ is the ring identity; $(00)_2$ the zero of the ring; multiplication by $(01)_2$ is the right shift operation; by distributivity $11 \cdot b_0 b_1 = 10 \cdot b_0 b_1 + 01 \cdot b_0 b_1 = b_0(b_1 \oplus b_0)$. Note that since $\rho_2(i) = 11$, if $\rho_1(x) = 1$, then either $\rho_2(x) = 10$ or $\rho_2(ix) = 10$. Similarly, modulo γ^3, the addition is

$$b_0 b_1 b_2 + b'_0 b'_1 b'_2 = (b_0 \oplus b'_0)(b_1 \oplus b'_1)(b_2 \oplus b'_2 \oplus (b_0 \cdot b'_0)),$$

where the $\cdot$ is the multiplication of the ring $\mathbb{Z}[i]/(\gamma)$, equivalently the "and" of two bits. The ring identity is 100, zero is 000. Multiplication of 010 is a right shit by 1 digit and multiplication of 001 is a right shit by 2 digits. The rest of the multiplication table can be obtained using distributativity. The additive inverse is

$$-(b_0 b_1 b_2) = b_0 b_1 (b_2 \oplus b_0).$$

By looking at the ρ_3 table, we see ρ_3 is not "stable" under taking complex conjugation, but instead, if $\rho_3(x) = abc$, then $\rho_3(x^\dagger) = ab\bar{c}$. Since $\rho_3(i) = 111$, if $\rho_1(x) = 1$, then there exists $k \in \{0,1,2,3\}$ such that $\rho_3(i^k x) = 100$.

Definition II.3. For $x \in \mathbb{Z}[i]$, if $\rho_1(x) = 0$, we say x is *even*, otherwise *odd*.

For example, $2+3i$ and $5+2i$ are odd, and $2+4i$ and $1+3i$ are even. In particular, in our context, by an even Gaussian integer we mean one that is divisible by γ, not necessarily by 2. However if $x \in \mathbb{Z}$, then x is even (odd) as an integer if and only if x is even (odd) as a Gaussian integer. Therefore, our definition of parity extends the usual one on the integers. The following lemmas are straightforward.

Lemma II.4. *Let $\alpha = a + bi \in \mathbb{Z}[i]$. Then α is odd if and only if $\|\alpha\|^2 = a^2 + b^2$ is an odd integer.*

Lemma II.5. *If $\alpha \in \mathbb{Z}[i]$ is odd, then $\alpha \equiv \pm 1, \pm i \,(\mathrm{mod}\ \gamma^3)$. In other words, $\alpha \equiv i^e \,(\mathrm{mod}\ \gamma^3)$ for some $e \in \{0,1,2,3\}$. Moreover, $\alpha \equiv i^e \,(\mathrm{mod}\ \gamma^2)$ for some $e \in \{0,1\}$.*

Lemma II.6. *If $\alpha, \beta \in \mathbb{Z}[i]$ satisfy $\alpha\gamma \equiv_{\gamma^{n+1}} \beta\gamma$, then $\alpha \equiv_{\gamma^n} \beta$.*

Let k be a denominator exponent (not necessarily the least) for $x \in \mathbb{D}[i]$, we define the residue function $\rho_n^k(x) = \rho_n(\gamma^k x)$. We call $\rho_n^k(x)$ the (n,k) residue of x. If k is a denominator exponent for a vector or matrix A, $\rho_n^k(A)$ is defined entry-wise.

Lemma II.7. *For $t \in \mathbb{D}[i]$, if $k = lde_\gamma(t) > 0$, then $\gamma^k t$ is odd.*

Lemma II.8. *For any $x \in \mathbb{D}[i]$ with denominator exponent k, $\rho_2^k(\bar{x}) = \rho_2^k(x)$, where $\bar{\cdot}$ is the complex conjugation.*

Lemma II.9. *For A a matrix with lde k, B another matrix of proper size with lde 0, then $\rho_n^k(AB) = \rho_n^k(A)\rho_n^0(B)$ and $\rho_n^k(BA) = \rho_n^0(B)\rho_n^k(A)$.*

This is because $\rho_n^k(\cdot)$ is a ring homomorphism and matrix multiplication only involves ring operations on matrix entries.

C. Action on residues

Suppose $\rho_2(x) = b_0 b_1$, then $\rho_2(ix) = b_0(b_1 \oplus b_0)$. We say i acts on $b_0 b_1$ by sending it to $b_0(b_1 \oplus b_0)$.

Given an n residue $b_0 b_1 ... b_{n-1}$, a right shift of it is an $n+1$ residue $0 b_0 b_1 ... b_{n-1}$. Given an $n+1$ residue $0 b_0 b_1 ... b_{n-1}$, a left shift is $b_0 b_1 ... b_n$. Note that we choose to only allow "well-defined" shifts in the sense no information is lost and no unknown is introduced. Note that the usual right shift is used previously when explaining the multiplication of $\mathbb{Z}[i]/(\gamma^2)$ and $\mathbb{Z}[i]/(\gamma^3)$. We use $LS(x)$ for the left shift of x and RS for the right shift.

Lemma II.10. *For $t \in \mathbb{D}[i]$, k a denominator exponent of t, we have*

$$\rho_{n+1}^{k+1}(t) = \rho_{n+1}^k(\gamma t) = RS(\rho_n^k(t)) \text{ and}$$

and if $\gamma^k t$ is even,

$$\rho_n^{k-1}(t) = \rho_n^k(t/\gamma) = LS(\rho_{n+1}^k(t)).$$

Given $x, y \in \mathbb{D}[i]$ with lde $k > 0$, we know $\rho_2^k(x) = 1b$ and $\rho_2^k(y) = 1b'$ for some $b, b' \in \{0,1\}$. So $\rho_2^k(x+y) = 1b + 1b' = 0(b \oplus b')$. Then $\rho_1^{k-1}(x+y) = LS(\rho_2^k(x+y)) = b \oplus b'$. Similarly, $\rho_1^{k-1}(x-y) = LS(\rho_2^k(x-y)) = LS(1b - 1b') = LS((1-1)(b-b')) = b \oplus b'$ (subtraction in $\mathbb{Z}/(2)$ coincides with $\oplus$, in other words the additive inverse of b is itself). Note that if $b \oplus b' = 1$, then $\mathrm{lde}(x \pm y)$ decreases by 1, and if $b \oplus b' = 0$, $\mathrm{lde}(x \pm y)$ decreases by at least 2 if $k \geq 2$. Since dividing t by γ obviously increases $\mathrm{lde}(t)$ by 1, we have for $x, y \in \mathbb{D}[i]$ with lde $k > 0$ and $\rho_3^k(x) = 10b$ and $\rho_3^k(y) = 10b'$,

$$\rho_2^k(K\begin{bmatrix} x \\ y \end{bmatrix}) = \rho_2^k(\begin{bmatrix} (x+y)/\gamma \\ (x-y)/\gamma \end{bmatrix}) = LS(\rho_3^k(\begin{bmatrix} x+y \\ x-y \end{bmatrix})) = \begin{bmatrix} 0(b \oplus b' \oplus 1) \\ 0(b \oplus b') \end{bmatrix}.$$

So we have $\mathrm{lde}(K\begin{bmatrix} x \\ y \end{bmatrix}) = k-1$ and one entry of it has lde $k-1$ and the other at most $k-2$ if $k > 2$.

In general for $x, y \in \mathbb{D}[i]$ with lde $k > 0$ and $\rho_3^k(x) = 1b_0b_1$ and $\rho_3^k(y) = 1b_0'b_1'$,

$$\rho_2^k(K\begin{bmatrix} x \\ y \end{bmatrix}) = \rho_2^k(\begin{bmatrix} (x+y)/\gamma \\ (x-y)/\gamma \end{bmatrix}) = LS(\rho_3^k(\begin{bmatrix} x+y \\ x-y \end{bmatrix})) = \begin{bmatrix} (b_0 \oplus b_0')(b_1 \oplus b_1' \oplus 1) \\ (b_0 \oplus b_0')(b_1 \oplus b_1') \end{bmatrix}.$$

If $\rho_3^k(x) = 0b_0b_1$ and $\rho_3^k(y) = 0b_0'b_1'$,

$$\rho_2^k(K\begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} (b_0 \oplus b_0')(b_1 \oplus b_1') \\ (b_0 \oplus b_0')(b_1 \oplus b_1') \end{bmatrix}.$$

If $\rho_3^k(x) = 1b_0b_1$ and $\rho_3^k(y) = 0b_0'b_1'$,

$$\rho_3^{k+1}(K\begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} 1(b_0 \oplus b_0')(b_1 \oplus b_1') \\ 1(b_0 \oplus b_0')(b_1 \oplus b_1') \end{bmatrix}.$$

If $\rho_3^k(x) = 0b_0b_1$ and $\rho_3^k(y) = 1b_0'b_1'$,

$$\rho_3^{k+1}(K\begin{bmatrix} x \\ y \end{bmatrix}) = \begin{bmatrix} 1(b_0 \oplus b_0')(b_1 \oplus b_1') \\ 1(b_0 \oplus b_0')(b_1 \oplus b_1' \oplus 1) \end{bmatrix}.$$

III. SPECIAL UNITARIES

A. Permutation matrices

Let $a_0a_1a_2a_3$ be a permutation of $\{0,1,2,3\}$. The permutation matrix $P_{a_0a_1a_2a_3}$ is the unitary matrix that maps e_i to e_{a_i}, where $\{e_i\}$ is the standard basis. All 24 permutation matrix are Clifford operators and they can be implemented by circuits of lenght at most 3, see Table I.

P	circuit	P	circuit	P	circuit
0123	identity	1023	$X_1 \cdot CX$	2103	$X_0 \cdot XC$
1203	$Ex \cdot X_0 \cdot XC$	2013	$Ex \cdot X_1 \cdot CX$	0213	Ex
3210	$X_1 \cdot X_0$	2310	$X_0 \cdot CX$	2130	$Ex \cdot X_1 \cdot XC$
3120	$Ex \cdot X_1 \cdot X_0$	1320	$Ex \cdot X_0 \cdot CX$	1230	$X_1 \cdot XC$
3012	$XC \cdot X_1$	0312	$Ex \cdot XC$	0132	CX
3102	$Ex \cdot CX \cdot X_0$	1302	$Ex \cdot X_0$	1032	X_1
3021	$Ex \cdot XC \cdot X_1$	0321	XC	0231	$Ex \cdot CX$
3201	$CX \cdot X_0$	2301	X_0	2031	$Ex \cdot X_1$

TABLE I. Circuit implementation of 24 permutation matrices.

B. Diagonal unitary matrices

The only diagonal unitary matrices with entries in $\mathbb{D}[i]$ (and $\mathbb{Z}[i]$) are of the form

$$\mathrm{diag}(i^a, i^b, i^c, i^d), \text{ where } a, b, c, d \in \{0, 1, 2, 3\}.$$

They can all be implemented by Clifford+CS circuits with length of at most 6 upto a global phase i^k, using at most 1 CS gate. This because

$$Z_0^{b_0} Z_1^{b_1} S_0^{b_2} S_1^{b_3} CZ^{b_4} CS^{b_5} \cdot i^k, b_i \in \{0, 1\}, k \in \{0, 1, 2, 3\}.$$

are all distinct and diagonal, and it is of cardinality 4^4 which equals to the cardinality of the set of diagonal unitary matices.

C. Generalized permutation unitary matrices

A generalized permutation unitary matrix with entries in $\mathbb{D}[i]$ is a matrix with the same nonzero pattern as a permutation matrix (there is exactly one nonzero entry in each row and each column), but the nonzero entry can be any number of unit length, i.e, powers of i. We will call such a matrix a generalized permutation. It is easy to see each 4×4 generalized permutation is product of a permutation matrix and a diagonal matrix, hence can be implemented by a circuit of lenght at most 9 containing at most 1 CS gate, and no K gate.

IV. THE EXACT SYNTHESIS ALGORITHM

The method used is similar to the work of Russell [38] for the ring $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ which is more complicated than and contains the ring here $\mathbb{Z}[\frac{1}{2}, i]$. Such residue analysis method is widely used in exact synthesis, e.g., in [3, 14, 15, 18, 31]. Note that, comparing to the ad hoc use of residues in each work, we have given in Section II a uniform way of dealing with residue of different rings using binnary residues, which, we conjecture, works for any ξ-ring proposed in [29].

We start with the main theorem.

A. Main theorems

Theorem IV.1. *For any $A \in U(\mathbb{D}[i])$ with lde k, there exists a circuit C implementing A, using at most $2k$ K gates, at most $2k+1$ CS gates, and at most $9(2k+1)+2k$ gates upto a global phase.*

Note that the proof (given later) is constructive, so it gives an algorithm for computing such circuit.

B. Optimality

Definition IV.2. Let D be a Clifford+CS circuit. Let $[D]$ be the unitary operator it implements. Define

- $\mathrm{rkc}(D)$ = the number of K gate that appears in D.
- $\mathrm{kc}(D) = \min\{\mathrm{rkc}(C) \mid C \in \mathcal{CIR} \text{ and } [C] = [D]\}$.
- $\mathrm{rcs}(D)$ = the number of CS gate that appears in D.
- $\mathrm{cs}(D) = \min\{\mathrm{rcs}(C) \mid C \in \mathcal{CIR} \text{ and } [C] = [D]\}$.
- $\mathrm{rlen}(D)$ = the number of gate that D contains.
- $\mathrm{len}(D) = \min\{\mathrm{rlen}(C) \mid C \in \mathcal{CIR} \text{ and } [C] = [D]\}$.

We say D is K-optimal, CS-optimal, and shortest if $\mathrm{rkc}(D) = \mathrm{kc}(D)$, $\mathrm{rcs}(D) = \mathrm{cs}(D)$, and $\mathrm{rlen}(D) = \mathrm{len}(D)$ repectively. We call $\mathrm{rkc}(D)$ and $\mathrm{rcs}(D)$ the K-count and CS-count of D repectively, and $\mathrm{kc}(D)$, $\mathrm{cs}(D)$ the optimal K-count and the optimal CS-count of D respectively.

Remark IV.3. Note that this calling differs with the other convention in e.g. [21], where they would call, e.g. $\mathrm{cs}(D)$ the CS-count of D. In other words, the other convention has no definition for "raw" K-count (rkc) or CS-count (rcs), and the "count" they have is only optimal.

Any Clifford+CS circuit D is an alternating sequence of Clifford part C_i and a CS gate

$$D = C_0 \cdot CS \cdot C_1 \cdot CS \cdot \ldots \cdot C_n. \tag{2}$$

By enumerating all 2-qubit Clifford circuit, we know C_i is implementable by at most 2 K gate and $\mathrm{lde}(C_i) \leq 2$. By subadditivity of lde, we have $\mathrm{lde}(D) \leq \mathrm{lde}(C_0) + \mathrm{lde}(CS) + \mathrm{lde}(C_1) \ldots \mathrm{lde}(C_n) \leq 2(n+1)$, that is, $\mathrm{lde}(D) \leq (2(\mathrm{rcs}(D) + 1)$ which implies

$$\mathrm{lde}(D) \leq 2(\mathrm{cs}(D) + 1).$$

Suppose 2 is CS-optimal, i.e., $\mathrm{cs}(D) = n$, We have $\mathrm{rkc}(D) \leq 2\,\mathrm{cs}(D) + 1)$ which implies

$$\mathrm{kc}(D) \leq 2(\mathrm{cs}(D) + 1).$$

Any Clifford+CS circuit D is also an alternating sequence of generalized permutation part P_i and a K_1 gate (by replacing all appearences of K_0 by $Ex \cdot K_1 \cdot Ex$):

$$D = P_0 \cdot K_1 \cdot P_1 \cdot K_1 \cdot \ldots \cdot P_n. \tag{3}$$

Each generalized permutation is implementable by at most 1 CS gate and uses no K gate. So we have $\mathrm{cs}(D) \leq \mathrm{rkc}(D) + 1$, which implies

$$\mathrm{cs}(D) \leq \mathrm{kc}(D) + 1.$$

We also have, by subadditivity of lde, that $\mathrm{lde}(D) \leq \mathrm{rkc}(D)$, which implies

$$\mathrm{lde}(D) \leq \mathrm{kc}(D).$$

Theorem IV.1 outputs D' such that $\mathrm{rkc}(D') \leq 2\mathrm{lde}(D)$ and $\mathrm{rcs}(D') \leq 2\mathrm{lde}(D) + 1$, which, together with the provious inequalities, implies

$$\mathrm{lde}(D) \leq \mathrm{kc}(D) \leq 2\mathrm{lde}(D) \tag{4}$$

$$\mathrm{lde}(D)/2 - 1 \leq \mathrm{cs}(D) \leq 2\mathrm{lde}(D) + 2 \tag{5}$$

From above equation, our synthesis algorithm is at most 2 times worse than the optimal K-count, and at most 4 times worse than the optimal CS-count (experiments show it always acheives the optimal CS-count, and better K-count than best-known, see Section V). Note that Inequality 5 is similar to Propsition 6 in [18] but slightly tigher.

1. *K optimality of Glaudell et al's algorithm [18]*

The work of Glaudell et al. didn't consider the K-count at all, but we can derive from their CS optimality that for a unitary A,

$$\mathrm{cs}(A) \leq \mathrm{kc}(A) \leq 2\,\mathrm{cs}(A) + 2.$$

This is because the output of their algorithm is of the form Equation 2 with the CS-count being optimal and the Clifford in between CS gate has K-count 1 or 2. Take a C_i that is in between CS gate and call it E. First, $\mathrm{kc}(E) \neq 0$, becuase then E is also a generalized permutation which contains no CS gate because it is Clifford. So the two CS in $CSECS$ will cancell decreasing the CS-count, whcih contradicts the optmality. The cancelling can be shown by direct calculation since the generalized permutations are finite, or using the semi-direct product structure that the permuations and diagonals together form. We can show $\mathrm{kc}(E) \leq 2$ by enumerating all two-qubit Cliffords.

So Glaudell et al's algorithm enjoys the same property as ours: the K-count is at most 2 times worse then optimal.

Remark IV.4. Note that exmperiments show that most of the case $\mathrm{lde}(A) < \mathrm{cs}(A)$ for random generated A. So our bound $\mathrm{kc}(A) \leq 2\mathrm{lde}(A)$ is strictly better in practice, and indeed experiments show that our method always acheives better K-count.

C. Complexity

The time complexity is $O(\mathrm{lde}(U))$.

D. Lemmas and proofs

We start with a number of lemmas. Since $\mathbb{Z}[i]$ and $\mathbb{D}[i]$ are subrings of $\mathbb{C}$, the usual properties of complex numbers, complex vectors, and complex matrices, such as complex conjugation (denoted by $\bar{\cdot}$), inner product, norm, and conjugate transposition (denoted by $\cdot^\dagger$) are naturally inherited. For example, for a vector $v \in \mathbb{D}[i]^n$, the *norm* of v is $\|v\| = \sqrt{v^\dagger v}$, where $v^\dagger$ is the conjugate transpose of v. Note that $\|v\|^2 = v^\dagger v \in \mathbb{D}$, since $v^\dagger v \in \mathbb{D}[i] \cap \mathbb{R}$. Similarly if $v \in \mathbb{Z}[i]$, then $v^\dagger v \in \mathbb{Z}$. A *unit vector* is a vector of norm 1.

If v is a vector, the notation v_j refers to its jth entry; entries are numbered starting from $j = 0$.

Lemma IV.5. *For any $A \in U(\mathbb{D}[i])$ with lde k, there exists permutation matrix L and R such that, $\rho_1^k(LAR)$ or $\rho_1^k((LAR)^T)$ equals one of the following*

$$(i)\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\end{bmatrix},\quad (ii)\begin{bmatrix}1&1&0&0\\1&1&0&0\\0&0&0&0\\0&0&0&0\end{bmatrix},\quad (iii)\begin{bmatrix}1&1&0&0\\1&1&0&0\\0&0&1&1\\0&0&1&1\end{bmatrix},$$

$$(iv)\begin{bmatrix}1&1&1&1\\1&1&1&1\\0&0&0&0\\0&0&0&0\end{bmatrix},\quad (v)\begin{bmatrix}1&1&0&0\\1&1&0&0\\1&1&1&1\\1&1&1&1\end{bmatrix},\quad (vi)\begin{bmatrix}1&1&1&1\\1&1&1&1\\1&1&1&1\\1&1&1&1\end{bmatrix}.$$

We also say A has such pattern and write $\mathrm{pat}(A)$ *for the pattern of A. Moreover, A has pattern (i) if and only if $\mathrm{lde}(A) = 0$, i.e., when A is a generalized permutation. A has pattern (iv), (v), and (vi) only if $\mathrm{lde}(A) > 1$.*

Before proving this lemma, we prove serveral useful lemmas.

Lemma IV.6. *Let v be a unit vector in $\mathbb{D}[i]^4$ with lde $k > 0$ then there exists a generalized unitary permutation matrix Q such that $\rho_1^k(Qv)$ equals $(1,1,0,0)^T$ or $(1,1,1,1)^T$. Moreover if $k > 1$ there exists a generalized unitary permutation matrix Q such that $\rho_2^k(Qv)$ equals to $(10,10,01,00)^T$ or $(10,10,10,10)^T$, and if $k = 1$, $\rho_2^1(Qv)$ equals to $(10,10,00,00)^T$.*

Proof. Let $w = \gamma^k v \in \mathbb{Z}[i]^4$. We first show w has an even number of odd entries. We have

$$\sum_{j=0}^{3} \|w_j\|^2 = \|w\| = |\gamma|^{2k}\|v\| = (\gamma\gamma^\dagger)^k \equiv 0 \,(\mathrm{mod}\ \gamma).$$

Therefore, there are an even number of $j \in \{0, ..., 3\}$ such that $\|w_j\|^2$ is odd. It follows by Lemma II.4 that an even number of w_j are odd.

We can find a permutation matrix P such that $\rho_2^k(Pv) = (1a, 1b, 0c, 0d)^T$ or $(1a, 1b, 1c, 1d)^T$. For the later, by muliplying a diagonal matrix, i.e., multiplying i

to the required entries, we can make the second digit all zero, i.e., $(10, 10, 10, 10)^T$.

For the former, by muliplying a diagonal matrix, we can make second digit of the odd entries zero, i.e., $(10, 10, 0c, 0d)^T$. Now applying K to the pair of odd entries we get $(00, 01, 0c, 0d)^T$ or $(01, 00, 0c, 0d)^T$. Note that multiplication of v and a unitary is still a uinit vector, so is applying K to the pair of odd entries. So $(01, 00, 0c, 0d)^T$ or $(00, 01, 0c, 0d)^T$ is a residue of a unit vector with lde $k-1$, then its one-digit $k-1$ residue must of the form $(1, 1, 0, 0)^T$ upto a permutation, that is $c = 1$ and $d = 0$ upto a permutation.

If $k = 1$, the only unit vectors are $\frac{1}{\gamma}(i^l, i^m, 0, 0)^T$ upto a permutation. □

Lemma IV.7. *Let u, v be two unit vectors in $\mathbb{D}[i]^4$ with lde $k > 0$ such that $u \perp v$, i.e., $u^\dagger v = 0$. Then there exists a generalized unitary permutation matrix Q such that $\rho_1^k(Q[u|v])$ equals*

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \text{ or } \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Proof. By Lemma IV.6, there can be only even number of 1s in each column. The only missing case is

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Then $\rho_1^k(u^\dagger v) = \sum_{j=0}^{3} \rho_1^k(\bar{u}_i v_i) = 1$ contradicts that $\rho_1^k(u^\dagger v) = \rho_1^k(0) = 0$. In other words, the number of indices such that at which the $(1, k)$ residue of u, v is 1 must be even. □

Proof of Lemma IV.5. Since the rows and columns of a unitary matrix are unit vectors and all row(column) vectors are perpendicular to each other. Lemma IV.6 and IV.7 forces the pattern to be (ii) - (vi) if $k > 0$. If $k = 0$, the only unitaries with lde 0 are the generalized permutations which has pattern (i). If $k = 1$, the only unit vectors are $\frac{1}{\gamma}(i^l, i^m, 0, 0)^T$ upto a permutation. So residue of such vectors cannot have four 1s. So Case (iv), (v), and (vi) only happens if the lde > 1. □

Lemma IV.8. *For any $A \in U(\mathbb{D}[i])$ with lde $k > 0$, there exist generalized permutations L, R, L', R' such that*

- *$lde(K_1 LAR) \leq k - 1$;*
- *or $lde(LARK_1) \leq k - 1$;*
- *or $lde(L'K_1 LARR'K_1) \leq k - 1$;*
- *or $lde(K_1 L'K_1 LARR') \leq k - 1$;*
- *or $lde(L'CKLAR) \leq k - 1$ (only happens when $lde(A) = 1$).*

Note that each $\leq$ above is actually $=$, but we don't need this. The proof proceeds by a case distinction over $\rho_1^k(A)$. Note that requiring $k > 0$ eliminates Case i. So start with Case ii.

1. Case ii

If $k > 1$, by Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 0b & 0\bar{b} \\ 01 & 0c & 0d & 0e \\ 00 & 0\bar{c} & 0f & 0g \end{bmatrix},$$

where $\bar{b} = b \oplus 1$ and b, c, d, e, f, g are indeterminates. We claim $a = 0, b = c = 1$. Taking inner product of the first two cloumn gives

$$10 + 1a + 00 + 00 = 00 \Rightarrow a = 0.$$

Taking inner product of the second and third row gives

$$01 + 0c + 00 + 00 = 00 \Rightarrow c = 1.$$

Similarly, we have $b = 1$. Now pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 01 & 00 \\ 01 & 01 & 0d & 0e \\ 00 & 00 & 0f & 0g \end{bmatrix}.$$

Moreover, since $k > 1$ and the third and fourth row (column) is of lde $k - 1 > 0$, $d = f$, $d = e$, $e = g$, i.e., $d = e = f = g$. Now pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 01 & 00 \\ 01 & 01 & 0d & 0d \\ 00 & 00 & 0d & 0d \end{bmatrix}.$$

Applying $K_1 \cdot S_1$ on the left gives Case iv.

If $k = 1$, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 00 & 00 \\ 10 & 1a & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix},$$

As before, we have $a = 0$. Apply $X_0 \cdot CK \cdot X_0$ on the left decreases the lde.

Remark IV.9. Note that the two generalized permutations we use to refine the pattern can be choosen to be CS-free. This is because the (2,k)-residue of the two rows (columns) of even entries are stable under multipicaton of i and we can introduce "dummy" i at a proper place to make a generalized permutation CS-free.

2. Case iii

If $k > 1$, by Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 0b & 0\bar{b} \\ 01 & 0c & 10 & 10 \\ 00 & 0\bar{c} & 10 & 1d \end{bmatrix},$$

Taking inner product of the first two cloumn gives

$$10 + 1a + 00 + 00 = 00 \Rightarrow a = 0.$$

Similarly $d = 0$. Taking inner product of the second and third row gives

$$01 + 0c + 0b + 0\bar{b} = 00 \Rightarrow c = 0.$$

Similary, we have $b = 0$. pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 00 & 01 \\ 01 & 00 & 10 & 10 \\ 00 & 01 & 10 & 10 \end{bmatrix}.$$

Applying $K_1 S_1$ on the left bring us to Case vi.

If $k = 1$. By Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 00 & 00 \\ 10 & 1a & 00 & 00 \\ 00 & 00 & 10 & 10 \\ 00 & 00 & 10 & 1d \end{bmatrix}.$$

As before, we have $a = 0, d = 0$. Applying K_1 on the left decrease the lde.

3. Case iv

This case only happens for $k > 1$. By Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 1a & 1b & 1c \\ 01 & 0d & 0e & 0f \\ 00 & 0\bar{d} & 0\bar{e} & 0\bar{f} \end{bmatrix}.$$

Taking inner product of the first two cloumn gives

$$10 + 1a + 00 + 00 = 00 \Rightarrow a = 0.$$

Similarly we have $b = c = 0$. Since the third row is of lde $k - 1 > 0$, either $d = e = f = 1$ or one of d, e, f is 1 and the rest are 0. For the later, upto a permutation of columns, we may assume $d = 1, e = f = 0$. Then, for both cases, right multiplication of K_1 decrease the lde.

Remark IV.10. As in Remark IV.9, the left generalized permutation can be choosen to be CS-free.

4. Case v

This case only happens for $k > 1$. By Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 1a & 00 & 01 \\ 10 & 1b & 10 & 1f \\ 10 & 1c & 1e & 10 \end{bmatrix},$$

We claim $a = 0, b = c = e = 1$. Taking inner product of the first two row gives

$$10 + 1a + 00 + 00 = 00 \Rightarrow a = 0.$$

Taking inner product of the first and third row gives

$$10 + 1b + 01 + 00 = 00 \Rightarrow b = 1.$$

Taking inner product of the second and fourth row gives

$$10 + 1c + 00 + 01 = 00 \Rightarrow c = 1.$$

Taking inner product of the second and third column gives

$$01 + 00 + 11 + 1\bar{e} = 00 \Rightarrow e = 1.$$

Similarly $f = 1$. Now pattern is

$$\begin{bmatrix} 10 & 10 & 01 & 00 \\ 10 & 10 & 00 & 01 \\ 10 & 11 & 10 & 11 \\ 10 & 11 & 11 & 10 \end{bmatrix},$$

Applying K_1 on the left bring us to Case iv.

5. Case vi

This case only happens for $k > 1$. By Lemma IV.6, upto multilying generalized permutations on the left and right, pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 1a & 1d & 1g \\ 10 & 1b & 1e & 1h \\ 10 & 1c & 1f & 1x \end{bmatrix},$$

Taking inner product of first two columns

$$a + b + c = 0 \Rightarrow a, b, c \text{ cannot be all 1.}$$

Upto a permutation, we maybe assume $a = 0$. Then $b = c$. Similarly $d = g$.

If $b = c = 1$, and $d = g = 1$, pat(A) is

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 11 & 11 \\ 10 & 11 & 1e & 1h \\ 10 & 11 & 1f & 1x \end{bmatrix},$$

Taking inner product of the first and third column gives $f = \bar{e}$. Similarly, $h = \bar{e}$ and $x = \bar{f}$. Now pat(A) is

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 11 & 11 \\ 10 & 11 & 1e & 1\bar{e} \\ 10 & 11 & 1\bar{e} & 1e \end{bmatrix}.$$

We prove this pattern cannot happen by looking at its (3,k)-residue. Upto multiplication of powers of -1 on the proper rows and columns, the (3,k)-residue refines to

$$\begin{bmatrix} 101 & 101 & 101 & 101 \\ 101 & 10x & 11u & 11a \\ 101 & 11y & 1ev & 1\bar{e}b \\ 101 & 11z & 1\bar{e}w & 1ec \end{bmatrix}.$$

Unitarity of A and the $k > 1$ requires that the any two rows (columns) are perperdicular to each other. In particular, by taking inner product of the first columns and the second, third column, we have

$$101 + 10x + 11y + 11z = 000 \implies x \oplus y \oplus z = 1$$
$$101 + 11u + 1ev + 1\bar{e}w = 000 \implies u \oplus v \oplus w = 1$$

Taking inner product of the second and thired column gives

$$101 + 11(u \oplus \bar{x}) + (1ev + 01e + 00\bar{y}) + (1\bar{e}w + 01\bar{e} + 00\bar{z})$$
$$= 000 \implies x \oplus y \oplus z \oplus u \oplus v \oplus w = 1, \text{ a contradiction.}$$

If $b = c = 0$, taking inner product of the first and third column gives $e = h$, similarly, $f = x$. So pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 1d & 1d \\ 10 & 10 & 1e & 1e \\ 10 & 10 & 1f & 1f \end{bmatrix},$$

Taking inner product of the first and third row gives $d \oplus e \oplus f = 0$, then at leat one of d, e, f is 0, upto a permutation of rows, we may assume $d = 0$. Similar analysis applies if $d = g = 0$. So, for either case, upto permutations of rows and columns pat(A) refines to

$$\begin{bmatrix} 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 \\ 10 & 10 & 1e & 1e \\ 10 & 10 & 1f & 1f \end{bmatrix},$$

Note that $e = f$, which is implied by taking inner product of the first and third column. So left multiplication of K_1 reduces the lde.

The transitions is summarized in Figure 1.

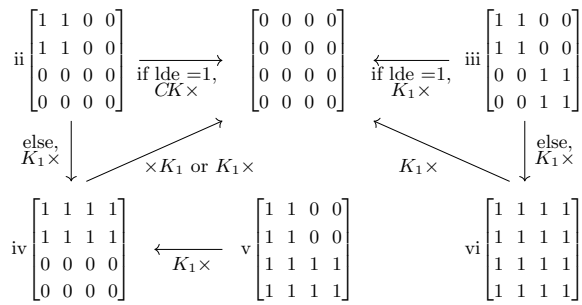

FIG. 1. Transitions between cases. The $K_1\times$ ($\times K_1$) annotated arrow means if A is of the pattern at the tail, then K_1LAR ($LARK_1$) is of the pattern at the head for appropritate choosen generalized permutations L and R. Similarly, for other annotations. Note that CK decomposes to two K gates as in Equation 1.

Proof of Theorem IV.1. Let $k = \text{lde}(A)$, by Lemma IV.8, possibly merging adjacent generalized permutaions, there exist generalized permutations $L_1, L_2, ..., L_p, R_1, R_2, ..., R_q$ such that

$$\text{lde}(L_pK_1L_{p-1}K_1...L_2K_1L_1AR_1K_1R_2K_1..., R_q) = 0$$

So there exists a generalized permutation P such that

$$L_pK_1L_{p-1}K_1...L_2K_1L_1AR_1K_1R_2K_1..., R_q = P$$

Note the rkc of left-handside $\leq 2k$. Then we have

$$A = L_1^{-1}iK_1L_2^{-1}...L_p^{-1}PR_q^{-1}iK_1...R_2^{-1}iK_1R_1^{-1}$$

Note that the rkc of the right handside is at most $2k$, after merging adjacent generalized permutaions, the number of generalized permutations is at most $2k + 1$. Each generalized permutation is implementable by at most 9 gates and at most 1 CS gate, upto a global phase. So we have a circuit that implements A, using at most 2k K gates, at most 2k +1 CS gates, and at most 9(2k +1)+2k gates upto a global phase. □

V. EXPERIMENT

We implemented our algorithm in Haskell and did an comparison with Glaudell et al.'s CS-optimal synthesis algorithm [18]. A unitary of given lde range is randomly genrated and feeded to both algorithms, and the output circuits are compared on metrics rcs, rkc.

The output of their algorithm is of the form Equation 2 with the CS-count being optimal. We added one step to their algorithm to keep the K-count as small as possible: for each C_i, we find an equivalent ciruit with minimum K-count (could be 0,1,2).

We generated randomly 1,351 two-qubit Clifford+CS operators with lde ranging from 2 to 58, available in VI. Figure 2 is the distribution of lde, which has mean 27 and median 28. Figure 3 is the distribution of the relative

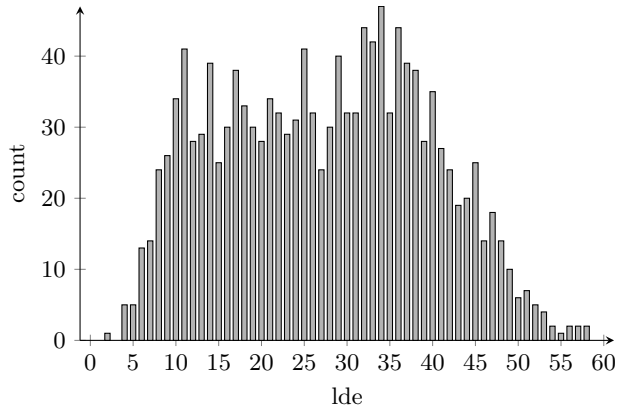

FIG. 2. Distribution of the least denominator exponent (lde) across 1,351 randomly generated two-qubit Clifford+CS operators. The lde ranges from 2 to 58 with mean 27 and median 28.

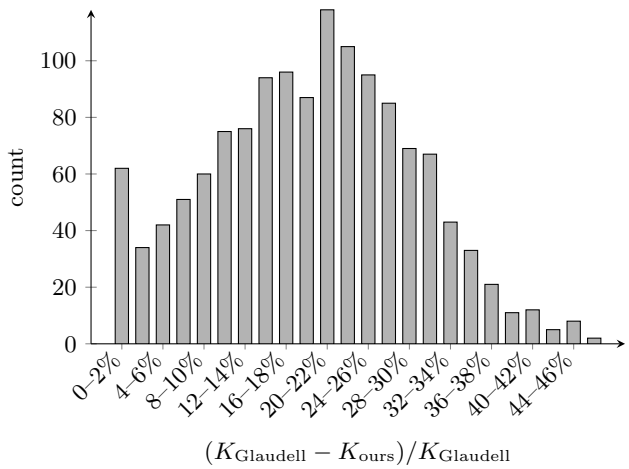

FIG. 3. Distribution of the relative K-count reduction $(K_{\text{Glaudell}} - K_{\text{ours}})/K_{\text{Glaudell}}$ across 1,351 randomly generated two-qubit Clifford+CS operators, binned in 2% increments. Our method improves on Glaudell et al.'s K-count in 95.4% of cases, with a mean reduction of 19.4%, a median of 19.8%, and a maximum of 47.7%.

K-count reduction, which has mean 19.4%, median of 19.8%, and maximum of 47.7%.

From the data, we also observe the following facts.

A. CS-count agreement

Across all 1,351 operators, our CS-count and Glaudell et al.'s CS-count are identical. Both algorithms are CS-optimal.

B. K-count improvement

Our algorithm produces a K-count less than or equal to that of Glaudell et al. in every case. It strictly improves the K-count in 95.4% of cases (1,289 out of 1,351), with a mean reduction of 19.4%, a median reduction of 19.8%, and a maximum reduction of 47.7%. Only 62 operators (4.6%) have identical K-counts under both methods; see Figure 3.

C. Empirical relations between CS-count and K-count, lde

The inequalities

$$CS\text{-count} \leq K_{\text{ours}} \text{ and } \text{lde} - 6 \leq CS\text{-count}$$

hold for every operator in the dataset. In particular, most of case, the CS-count is greater than or equal to the lde, but not always. Out of 1,351 operators:

- CS-count $>$ lde: 1,283 cases (95.0%),
- CS-count $=$ lde: 45 cases (3.3%),
- CS-count $<$ lde: 23 cases (1.7%).

In the 23 cases where CS-count $<$ lde, the deficit is usually 1 (i.e. CS-count $=$ lde $-$ 1), with the largest gaps being lde $=$ 38, CS-count $=$ 32 and lde $=$ 20, CS-count $=$ 18.

D. Source of improvement

Two tight linear relationships emerge from the data. First, the K-count of Glaudell et al. is well approximated by twice the CS-count:

$$K_{\text{Glaudell}} \approx 2 \cdot CS\text{-count}.$$

The residual $K_{\text{Glaudell}} - 2 \cdot CS$-count lies in $\{-4, -3, -2, -1, 0, 1, 2\}$ for all 1,351 operators, with 99.3% of values in $\{-4, \ldots, 0\}$. Second, our K-count is well approximated by twice the lde:

$$K_{\text{ours}} \approx 2 \cdot \text{lde}.$$

The residual $K_{\text{ours}} - 2 \cdot \text{lde}$ equals -1 or 0 for 80.2% of operators, and satisfies $|K_{\text{ours}} - 2 \cdot \text{lde}| \leq 2$ for 99.6% of operators. These two relationships are also reflected in the Pearson correlations: the correlation between lde and K_{ours} is $r = 0.9987$, while the correlation between CS-count and K_{Glaudell} is $r = 0.9991$.

Since $K_{\text{Glaudell}} \approx 2 \cdot CS$-count and $K_{\text{ours}} \approx 2 \cdot \text{lde}$, the absolute K-count saving is approximately $2 \cdot (CS\text{-count} - \text{lde})$. In other words, the improvement is governed by the gap between the CS-count and the lde. The relative reduction percentage is roughly stable across lde values, hovering around 19–21% for lde between 5 and 40, and tapering off for very small or very large lde.

VI. CODE AND DATA AVAILABILITY

VII. CONCLUSION AND FUTURE WORK

In conclusion, we proposed an efficient and near-optimal algorithm for the exact synthesis of two-qubit Clifford+CS circuits from a given unitary matrix. In theory, we proved the CS-count is at most 4 times worse than optimal, but in practice, our method always acheives the optimal CS-count. In theory, we proved the K-count is at most 2 times worse than optimal, and in practice, our method always acheives the better K-count than best-known.

The algorithm of Li et al. [31] outputs a circuit that is at most 10 times worse than the optimal T-count. We believe our method can be used to improve the bound 10.

VIII. ACKNOWLEDGEMENTS

The authors acknowledge support from the National Natural Science Foundation of China under Grant 92465202.

We have used Claude Opus 4.6 for

- creating four references [11, 16, 29, 30] and their one-line summary in the related work section.
- improving the wording of the paper.
- generating Haskell code: the ParseData.hs (for parsing data and drawing distribution diagrams) and CliffordCS.hs (the implementation of Glaudell et al.'s algorithm [18]).
- drafting Section VI, except for the first two paragraphs, with proper guide from the authors and later major editing by the authors.

[1] N. Abdessaied, M. Soeken, and R. Drechsler. Quantum circuit optimization by Hadamard gate reduction. In *Reversible Computation*, pages 149–162, 2014.

[2] M. Amy, A. N. Glaudell, S. Kelso, W. Maxwell, S. S. Mendelson, and N. J. Ross. Exact synthesis of multiqubit Clifford-Cyclotomic circuits, 2024. arXiv:2311.07741.

[3] M. Amy, A. N. Glaudell, S. M. Li, and N. J. Ross. Improved synthesis of Toffoli-Hadamard circuits, 2023. arXiv:2305.11305.

[4] M. Amy, A. N. Glaudell, and N. J. Ross. Number-theoretic characterizations of some restricted Clifford+T circuits. *Quantum*, 4:252, Apr 2020.

[5] M. Amy, D. Maslov, and M. Mosca. Polynomial-time T-Depth optimization of Clifford+T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1476–1489, 2014.

[6] X. Bian. EucDomain. `https://github.com/onestruggler/EucDomain`.

[7] A. Clement. Private communications.

[8] N. de Beaudrap, X. Bian, and Q. Wang. Fast and effective techniques for T-count reduction via spider nest identities. In *15th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2020)*, volume 158 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 11:1–11:23, 2020.

[9] N. de Beaudrap, X. Bian, and Q. Wang. Techniques to reduce $\pi/4$-parity-phase circuits, motivated by the ZX calculus. *Electronic Proceedings in Theoretical Computer Science*, 318:131–149, May 2020.

[10] L. Dinh and N. J. Ross. Contributions to the theory of Clifford-cyclotomic circuits. *Electronic Proceedings in Theoretical Computer Science*, 426:231–246, Aug. 2025.

[11] S. Forest, D. Gosset, V. Kliuchnikov, and D. McKinnon. Exact synthesis of single-qubit unitaries over Clifford-cyclotomic gate sets. *Journal of Mathematical Physics*, 56(8):082201, 2015.

[12] V. Gheorghiu, M. Mosca, and P. Mukhopadhyay. A (quasi-)polynomial time heuristic algorithm for synthesizing T-depth optimal circuits. 1 2021. arXiv:2101.03142.

[13] V. Gheorghiu, M. Mosca, and P. Mukhopadhyay. T-count and T-depth of any multi-qubit unitary. 10 2021. arXiv:2110.10292.

[14] B. Giles and P. Selinger. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A*, 87(3):032332 (7 pages), 2013.

[15] B. Giles and P. Selinger. Remarks on Matsumoto and Amano's normal form for single-qubit Clifford+T operators. 2013. arXiv:1312.6584.

[16] A. N. Glaudell, M. Jarret, S. Klein, S. S. Mendelson, T. C. Mooney, and M. Tian. High-performance exact synthesis of two-qubit quantum circuits, 2026.

[17] A. N. Glaudell, N. J. Ross, and J. M. Taylor. Canonical forms for single-qutrit Clifford+T operators. *Annals of Physics*, 406:54–70, 2019.

[18] A. N. Glaudell, N. J. Ross, and J. M. Taylor. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information*, 7(1):103, Jun 2021.

[19] A. N. Glaudell, N. J. Ross, J. van de Wetering, and L. Yeh. Qutrit Metaplectic Gates Are a Subset of Clifford+T. In *17th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2022)*, volume 232 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:15, 2022.

[20] A. N. Glaudell, N. J. Ross, J. van de Wetering, and L. Yeh. Exact synthesis of multiqutrit Clifford-cyclotomic circuits. *Electronic Proceedings in Theoretical Computer Science*, 406:44–62, Aug. 2024.

[21] D. Gosset, V. Kliuchnikov, M. Mosca, and V. Russo. An algorithm for the T-count. *Quantum Information and Computation*, 14(15–16):1261–1276, 2014.

[22] L. E. Heyfron and E. T. Campbell. An efficient quantum compiler that reduces T count. *Quantum Science and Technology*, 4(1):015004, sep 2018.

[23] K. Ireland and M. Rosen. *A Classical Introduction to Modern Number Theory*. Graduate Texts in Mathematics 84. 1982.

[24] A. Jain, A. R. Kalra, and S. Prakash. A normal form for single-qudit Clifford+T operators, 2020. arXiv:2011.07970.

[25] A. R. Kalra, M. Saikia, D. Valluri, S. Winnick, and J. Yard. Multi-qutrit exact synthesis, 2024. arXiv:2405.08147.

[26] A. Kissinger and J. van de Wetering. Reducing the number of non-Clifford gates in quantum circuits. *Phys. Rev. A*, 102:022406, Aug 2020.

[27] V. Kliuchnikov. Synthesis of unitaries with Clifford+T circuits, 2013. arXiv:1306.3200.

[28] V. Kliuchnikov, D. Maslov, and M. Mosca. Fast and efficient exact synthesis of single qubit unitaries generated by Clifford and T gates. *Quantum Information and Computation*, 13:607–630, June 2013.

[29] V. Kliuchnikov and S. Schönnenbeck. Multi-qubit circuit synthesis and Hermitian lattices, 2024. arXiv:2405.19302.

[30] V. Kliuchnikov and J. Yard. A framework for exact synthesis, 2015. arXiv:1504.04350.

[31] L. Li, C. Guo, Q. Li, and X. Sun. Fast exact synthesis of two-qubit unitaries using a near-minimum number of T gates. *Phys. Rev. A*, 107:042424, Apr 2023.

[32] K. Matsumoto and K. Amano. Representation of quantum circuits with Clifford and $\pi/8$ gates, 2008. arXiv:0806.3834.

[33] M. Mosca and P. Mukhopadhyay. A polynomial time and space heuristic algorithm for T-count. 6 2020. arXiv:2006.12440.

[34] P. Mukhopadhyay. Synthesis of V-count-optimal quantum circuits for multiqubit unitaries. *Physical Review A*, 109:052619, 5 2024.

[35] P. Mukhopadhyay. Synthesizing Toffoli-optimal quantum circuits for arbitrary multi-qubit unitaries. 1 2024. arXiv:2401.08950.

[36] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. 2002.

[37] S. Prakash, A. Jain, B. Kapur, and S. Seth. Normal form for single-qutrit Clifford+T operators and synthesis of single-qutrit gates. *Phys. Rev. A*, 98:032304, Sep 2018.

[38] T. Russell. The exact synthesis of 1- and 2-qubit Clifford+T circuits, 2014. arXiv:1408.6202.

[39] P. Selinger. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science*, 11(2:10):1–17, 2015.